- You can use the `IHostedService` interface to run tasks in the background of your ASP.NET Core apps. Call `AddHostedService<T>()` to add an implementation `T` to the DI container. `IHostedService` is useful for implementing long-running tasks.
- Typically, you should derive from `BackgroundService` to create an `IHostedService`, as this implements best practices required for long-running tasks. You must override a single method, `ExecuteAsync`, that is called when your app starts. You should run your tasks within this method until the provided `CancellationToken` indicates that the app is shutting down.
- You can create DI scopes manually using `IServiceProvider.CreateScope()`. This is useful for accessing scoped lifetime services from within a singleton lifetime component, such as from an `IHostedService` implementation.
- A worker service is a .NET Core application that uses the generic `IHost` but doesn't include the ASP.NET Core libraries for handling HTTP requests. It generally has a smaller memory and disk footprint than an ASP.NET Core equivalent.
- Worker services use the same logging, configuration, and DI systems as ASP.NET Core apps. However, they don't use the `WebApplicationBuilder` minimal hosting APIs, so you must configure your app using the generic host APIs. For example, configure your DI services using `IHostBuilder.ConfigureServices()`.